

A study of boundary layers in laminar and turbulent flows with computational fluid dynamics

Alec Stanley

April 2026

1 Introduction

Computational fluid dynamics (CFD) is an essential tool used extensively in the aerospace industry. It allows engineers to simulate the behaviour of flow over complicated structures that have no analytical solutions, predicting flows based on the governing equations of conservation of mass, momentum, and energy¹ [1]. This is far more cost-effective than constructing a dynamically similar flow. Most established codes are closed-source and too expensive for most users [2], however for this task the open-source SU2 application was used. This report focuses on the behaviour of boundary layers across three different flow regimes.

The three cases analysed were an incompressible laminar boundary layer over a flat plate, an incompressible turbulent boundary layer over a flat plate, and a compressible laminar flow over a circular cylinder in two dimensions. We analysed boundary layer thicknesses, wall shear stress, velocity, and temperature profiles, comparing the simulation values to theory where applicable. The study contrasts the attached flat plate flow and the boundary layer separation observed in the cylinder case, highlighting the differences in flow conditions between each case.

2 Results and Discussion

2.1 Laminar Flat Plate

In the case of laminar flow over a flat plate, we would expect nicely ordered fluid layers (streamlines) with a thin boundary layer arising due to the viscous effects. We have assumed a flat plate incident on $x = 0$ with length 1ft, or 0.3048m. Figure 1 shows the velocity field of the flow over the flat plate, separated into horizontal and vertical velocity components. There is a very clear boundary layer developing over the flat plate as indicated predominantly by the change in horizontal velocity. This arises, as expected, due to the no-slip boundary condition where the horizontal and vertical velocity at the boundary $y = 0$ must be zero. An artifact of this condition is a very discontinuous jump at $x = 0$ where the flow must jump from U_0 , the free-stream velocity, to zero. This effect is more clearly shown in the vertical velocity diagram in Figure 1. We would expect vertical velocity to be close to zero for most of the flow, slightly increasing within the boundary layer. What we see is that this is mostly true, except for the discontinuous infinite flow at $x = 0, \forall y$.

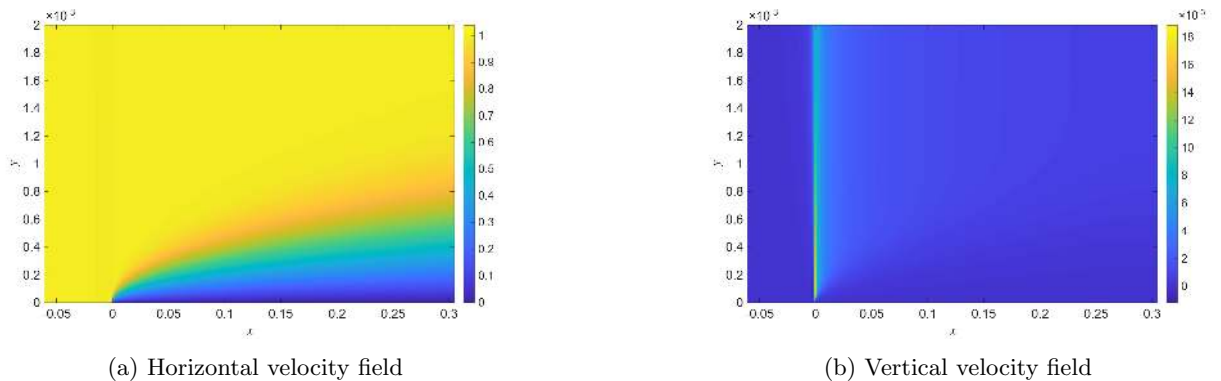


Figure 1: Velocity fields for a laminar flat plate with wall temperature of 148.81K.

¹The Navier-Stokes equations

There are many ways to define the thickness of the boundary layer. As observed, it is not a clear boundary line, where all points above are outside the boundary and all points below are inside the boundary. Instead, we see a continuous and gradual shift from the boundary layer velocity profile to the free-stream conditions. To define a boundary we must get creative.

A first step to define such a boundary is by defining a fixed point, say 99% of the free-stream velocity, where the boundary is defined as the height where the flow's velocity is 99% of that of the free-stream's. This is δ_{99} , or just simply δ . In our SU2 output, the velocity has already been normalised such that the free-stream velocity is 1.0. Figure 2 clearly shows the boundary layer thickness δ .

Another method to define the boundary layer is using the displacement thickness δ^* , defined as the distance, or height, a streamline is displaced upward by the presence of a boundary layer [3]. This is given mathematically by,

$$\delta^* = \int_0^{y_1} 1 - \frac{\rho u}{\rho_0 u_0} dy, \quad (1)$$

which is proportional to the missing mass flow due to the presence of the boundary layer.

The final thickness definition we will explore is the momentum thickness θ , which is proportional to the decrease in momentum flow due to the presence of the boundary layer [3]. It is given by,

$$\theta = \int_0^{y_1} \frac{\rho u}{\rho_0 u_0} \left(1 - \frac{u}{u_0}\right) dy, \quad (2)$$

which physically represents the height of a streamtube which is carrying the 'missing' momentum flow at free-stream conditions [3].

It is expected that the boundary layer thickness is the largest thickness, followed by the displacement thickness, followed by the momentum thickness, as can be determined simply by comparing the thickness definitions. We do see this in our results. Another artefact we see is how the displacement and momentum thickness start to increase before the plate even begins, highlighting a difference between theoretical and computational fluid dynamics. Theoretically speaking, there should be no boundary forming before the plate, as the flow is equal to the free stream velocity everywhere [3]. This is because the standard boundary layer equations used are parabolic, depending only on downstream conditions. They do not allow for upstream propagation. In contrast, the full Navier-Stokes equations are elliptic in nature, allowing for flow properties to propagate in all directions, including upstream [4].

All thickness lines (δ_{99} , δ^* , and θ) are shown in Figure 2.

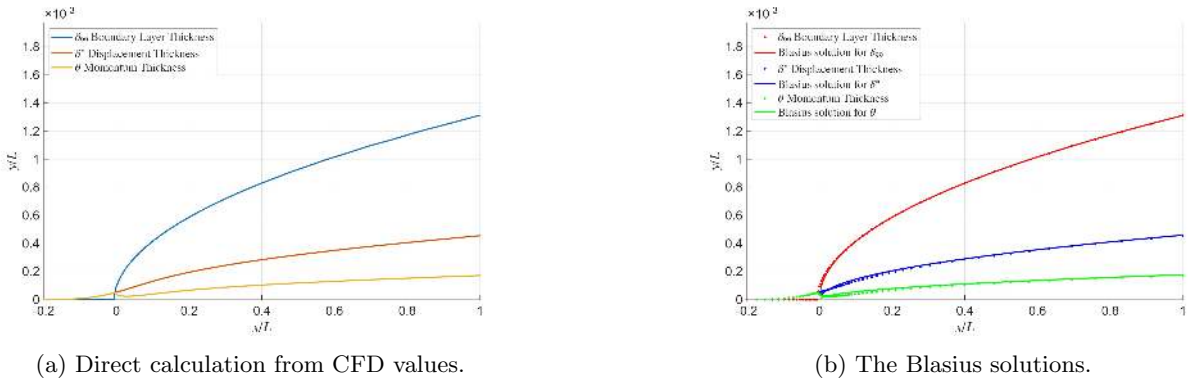


Figure 2: The boundary layer thickness, the displacement thickness, and the momentum thickness for laminar flow over a flat plate (left) and their theoretical Blasius solutions (right).

To determine how well the results for the boundary layer thickness agree with the literature, we can look to the Blasius solutions. The Blasius equation and its solutions are derived from Prandtl's boundary layer equations simplified for laminar flow over a flat plate, and are highly applicable to our case study. The Blasius solutions used in this report are taken from White [4]. The solution for boundary layer thickness is

$$\delta_{99} = \frac{4.91x}{\sqrt{\text{Re}_x}}. \quad (3)$$

The solution for displacement thickness is

$$\delta^* = \frac{1.721x}{\sqrt{\text{Re}_x}}. \quad (4)$$

The solution for momentum thickness is

$$\theta = \frac{0.664x}{\sqrt{\text{Re}_x}}. \quad (5)$$

From Figure 2 (right) we can see that the numerical CFD results very closely align with the theoretical Blasius solutions on the order of $x^{1/2}$ with a high degree of accuracy.

The friction coefficient of a fluid is, in general, given as

$$C_f = \frac{\tau_w}{q}, \quad (6)$$

where τ_w is the wall shear stress and q is the dynamic pressure.

$$\tau_w = \mu \frac{du}{dy} \Big|_{y=0}, \text{ and } q = \frac{1}{2} \rho U_0^2, \text{ where } U_0 \text{ is the free-stream velocity.}$$

If we evaluate this at all points along the flat plate we obtain Figure 3, which clearly shows the relationship between fluid friction against the wall and location along the plate. There are some notable features about this plot. We first notice that there is a large initial value at $x = 0$. These artefacts are unavoidable in CFD analysis of this nature. Due to the abrupt change in boundary condition, the instantaneous value of du at $x = 0$ is very high, resulting in a very large coefficient in Equation 6.

To compare this to literature, we know that the Blasius solution for the friction coefficient over a flat plate is

$$C_f = 0.664 \cdot \text{Re}_x^{-1/2}. \quad [2] \text{ and } [4] \quad (7)$$

This is also equal to the dimensionless momentum thickness (Equation 5).

We can see how our results compare to the Blasius literature in Figure 16 [2]. The CFD agrees with the theory for fluid friction at the boundary wall.

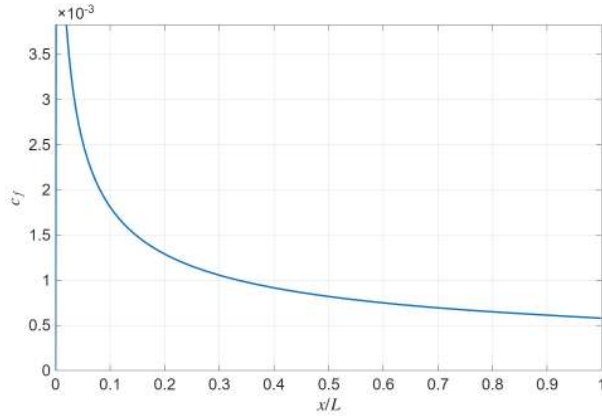


Figure 3: A plot depicting the streamwise friction coefficient of the flow against the wall.

The total drag coefficient of the fluid is given by

$$C_D = \frac{1}{L} \int_0^L C_f(x) dx. \quad (8)$$

The non-dimensional drag coefficient evaluates to 1.072×10^{-3} . The Blasius solution for the drag coefficient is defined as

$$C_D = \frac{1.328}{\sqrt{\text{Re}}}, \quad [4] \quad (9)$$

which evaluates to 1.164×10^{-3} using a Reynolds number of 1 301 233.166 [2]. The simulation agrees with the theory.

In a real experimental case, we would expect the behaviour of these boundary layers to eventually end. In real life we observe a transition to turbulence. As x continues to increase, the local Reynolds number eventually reaches a critical value where the smooth laminar boundary layer transitions to a turbulent one. The flow in this regime is much less predictable and in the context of CFD usually needs to be analysed using time-averaged streamflows, as is analysed in this report.

A good test to see if our flow is truly laminar is to check if it is self similar. This is a check to see if the flow retains its shape at all positions if normalised appropriately [3]. To check this, we can normalise the velocity profiles against the boundary layer thickness, plotting y/δ_{99} against u , as seen in Figure 4.

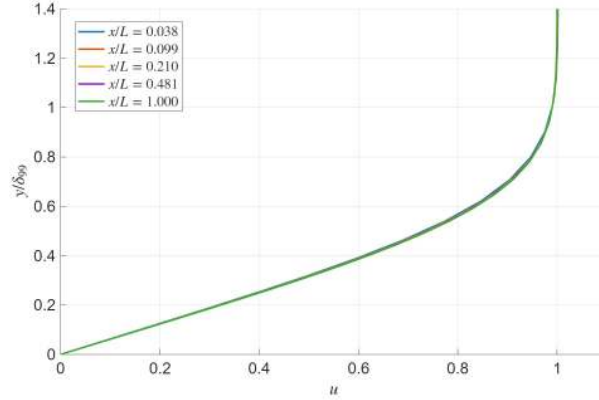


Figure 4: A plot depicting the normalised velocity profiles of the boundary layer flow, evaluated at various intervals along the flat plate.

What Figure 4 tells us is that, no matter what point along the flat plate we choose, the velocity profile normalised to the boundary layer thickness is the same. This means the flow retains the same shape as it moves across the plate. This is a highly useful characteristic that would allow us to move from analysing a two-dimensional flow to a simpler one-dimensional flow.

In the case of laminar flow over a flat plate, we are interested in the heat flow across the fluid. To explore this, we have developed three different cases. In each case, the free-stream temperature is kept at 297.62 K. In one case, we chose a cooler temperature, to be 148.81 K. In the second case we chose the wall temperature to be equal to the free-stream temperature. In the third case, we chose a hotter temperature, to be 350 K. The results of this experiment are shown in Figure 5. In each case, we would expect our boundary conditions to be kept. Specifically, the flow temperature at the wall would be locked to the wall temperature, and then as y moves beyond the boundary layer thickness it should approach the free stream temperature. We would also observe a smooth transition in temperature. This is what we observe.

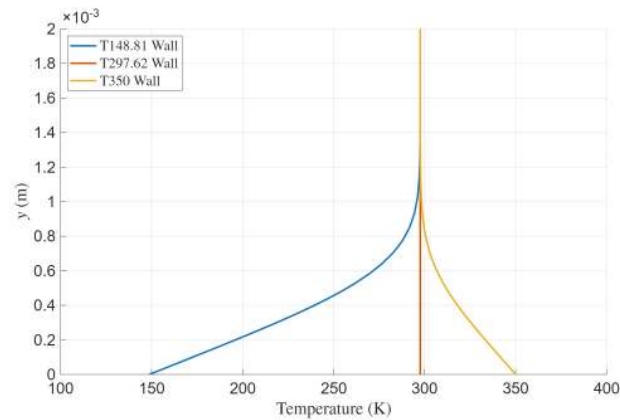


Figure 5: A plot depicting the temperature profile of the flow over a flat plate at an arbitrary point, chosen to be 0.147m. This was repeated for three different wall temperatures.

Evidently, in the case of $T_{\text{wall}} = T_{\infty}$ we observe a straight line, as predicted by the expectation that the temperature would not deviate in such a case. The behaviour for the temperature gradient is similar to what we observe for the boundary

layer streamwise velocity, and based on these results the temperature gradient exhibits boundary-layer-like behaviour, which is more commonly referred to as the thermal boundary layer. The thermal boundary layer is demonstrated to have similar thickness to the velocity boundary layer and develops in a similar way.

2.2 Turbulent Flat Plate

Unlike the smooth layered streamlines seen with laminar flow, turbulent flow is characterised by randomly fluctuating parameters. It is generally less understood [5], and involves superimposing erratic motion onto a mean flow (streamlines). Turbulent flow is characteristic of a high Reynolds number [5], and the Reynolds number used for our CFD is 10×10^6 . In our study of the boundary layer, we find that in turbulent flow, the viscous effects are much greater than the inviscid effects, and far greater than in the laminar case. Observe Figure 6.

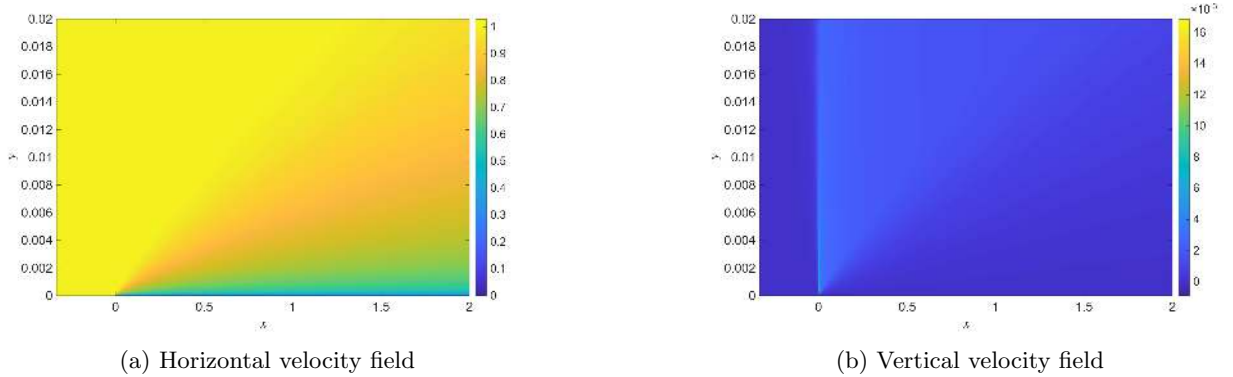


Figure 6: Velocity fields for a turbulent flat plate.

In the velocity fields for the turbulent case, the first artefact we notice is far more spread in the boundary layer. There is a much less defined boundary. We observe the same discontinuity at $x = 0$, as it is an unavoidable artefact of the CFD, however a much smoother and physically higher spread of velocity, which is easier seen in the horizontal velocity field. We also notice that the discontinuity is less harsh than in the laminar case, which arises due to the time-averaged nature of the turbulent CFD. The erratic fluid fluctuations in the flow are superimposed onto streamlines, leading to a characteristic smoothing effect in the final data.

Another notable feature in the turbulent case is that the boundary layer appears to be much thicker than the laminar case. This is an expected phenomenon since the viscous effects are much greater in turbulent flow. As a result, there is a much greater transfer of momentum over a greater distance, inevitably leading to a higher boundary layer of slowed fluid. We also observe that, like the laminar case, the no-slip boundary condition is conserved and the velocity still approaches the free-stream velocity sufficiently far from the wall.

Figure 7 reveals the boundary layer thickness, the displacement thickness, and the momentum thickness for the turbulent case. The most notable difference here is that the boundary layer thickness is far greater than in the laminar case. While the displacement and momentum thickness are of the same order ($\sim 0.2 - 0.4 \cdot 10^{-3}$) as the laminar case, the boundary layer thickness has been superimposed on a greater scale to sufficiently see all three boundaries. As discussed, this is an expected phenomenon. This occurs due to the turbulent mixing pulling higher velocity fluid down closer to the wall, meaning the 'missing mass' (δ^*) and 'missing momentum' (θ) represent a smaller fraction of the boundary layer thickness compared to laminar flow. The calculations for the thickness values are already referenced in Equations 1 and 2.

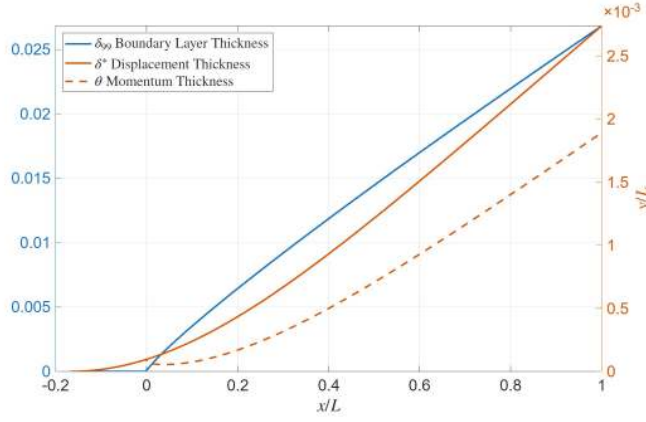


Figure 7: The boundary layer thickness, the displacement thickness, and the momentum thickness for turbulent flow over a flat plate.

The empirical solutions of the boundary layer thicknesses in the turbulent regime are on the order of $x^{4/5}$ [4], according to Prandtl's original 1927 power law correlations. More modern correlations use $x^{6/7}$, yet regardless, a turbulent boundary layer grows much more rapidly than a laminar boundary layer due to the enhanced mixing of the flow.

We can also observe the friction coefficient of the fluid as it moves across the flat plate, in a similar fashion to the laminar case. See Figure 8.

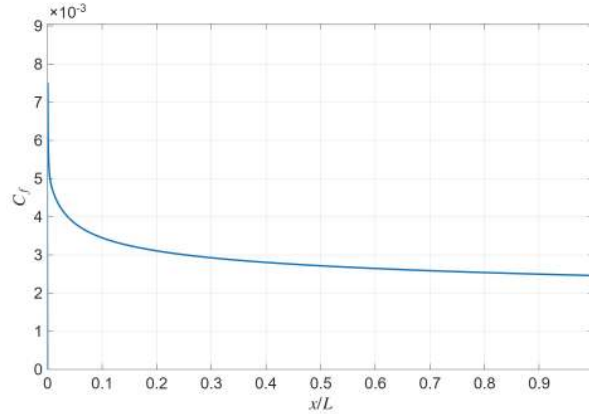


Figure 8: A plot depicting the streamwise friction coefficient of the turbulent flow.

What we see with the friction coefficient is a similar trend to that of the laminar case, albeit ≈ 2.5 times greater. It follows a steeper, more complex power law than the straightforward laminar Blasius solution². This is an expected result [5] where the turbulent skin friction is substantially higher than the laminar skin friction due to the presence of greater shear forces. By Equation 6 and 8 we can also determine the drag coefficient of the flow. This has a value of 2.870×10^{-3} , which is higher than the laminar drag coefficient. This arises again due to the turbulent mixing at the wall causing a higher velocity gradient. The empirical solution [4] for the drag coefficient (the 'wall wake law') in the turbulent regime is

$$C_D = \frac{0.031}{Re^{1/7}}, \quad (10)$$

which yields 3.1×10^{-3} , agreeing very well with the CFD result.

Interestingly, when we try to normalise the streamwise velocity profile at various intervals along the flat plate to determine the self similarity of the flow, we see some notable differences with the laminar case. Previously, we saw that even when the flow was close to 0 or 1 (normalised x-coordinate), the flow was almost perfectly self similar. In the turbulent case, this is clearly not the case. The flow close to $x = 0$ is evidently an outlier. The rest of the test profiles much more closely conform to a self similar profile. The general expectation for turbulent flow over a flat plate is that it is self-similar under the right pressure conditions and Reynolds number, subject to the constraints of the CFD. What we find is a case similar to the

²Since the Blasius equation only applies for the laminar case, we resort to empirical correlations for the comparison to literature results.

aforementioned issues when working over discontinuities in CFD. Since we are taking time-averaged streamlines rather than simple laminar flows, the smoothing effect over the start of the flat plate is so extreme that it alters the shape of the flow itself close to that point. From this specific simulation, we can only say that the flow is self similar if it is sufficiently far from the start of the flat plate.

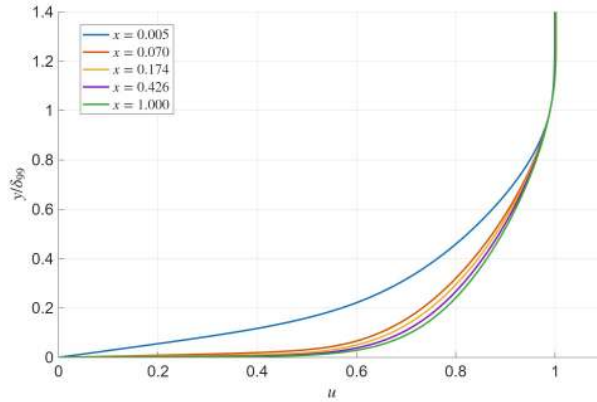


Figure 9: A plot depicting the normalised velocity profiles of the turbulent boundary layer flow at various intervals along the flat plate.

2.3 Laminar Cylinder

Flow around a cylinder³ in two dimensions for Reynolds numbers less than about 46 is steady and symmetric [2]. In our analysis, a Reynolds number of 40 was used. To understand the nature of the flow we are dealing with, refer first to Figure 10 which displays the pressure field around the cylinder. We can very clearly see the stagnation point that lies directly on the front of the cylinder, at an angle of 180° , where we would expect. We also see two equal points of lowest pressure occurring at the top and bottom of the cylinder, where the flow is traveling the fastest. We also observe that the pressure does not return to the free-stream pressure in the wake immediately following the cylinder, as expected with flow separation that has occurred in this flow regime [4].

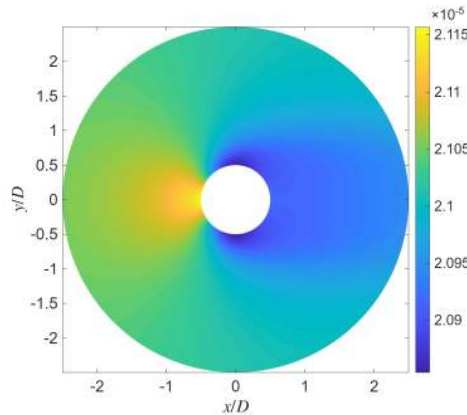


Figure 10: Pressure contour plot of laminar flow over a 2D cylinder with low Reynolds number.

Similar to how we did for the flat plate cases, we can look at the horizontal and vertical velocity fields in Figure 11. These are quite different to the flat plate case. The first thing to notice is the precise symmetry exhibited in the simulation. We see a clear stagnation point in the x -velocity to the left of the cylinder, and very low horizontal velocity in the turbulent wake after flow separation. We also see a spike in vertical velocity as the flow moves around the cylinder. Due to the subsonic nature of the flow, we see much more upstream propagation in this case due to the flow being fully continuous (unlike the flat plate case) and having a strong stagnation point.

Flow separation arises purely due to viscous effects. In the inviscid case, we would not see flow separation at all. The flow would perfectly reattach symmetrically about the cylinder. The pressure recovery would be nearly perfect and the total

³In this section, all angles are defined relative to the positive x -axis unless stated otherwise.

drag would be almost zero. However, in the viscous case we are dealing with the formation of a highly viscous boundary layer, which induces flow separation due to an adverse pressure gradient as the flow moves over the cylinder.

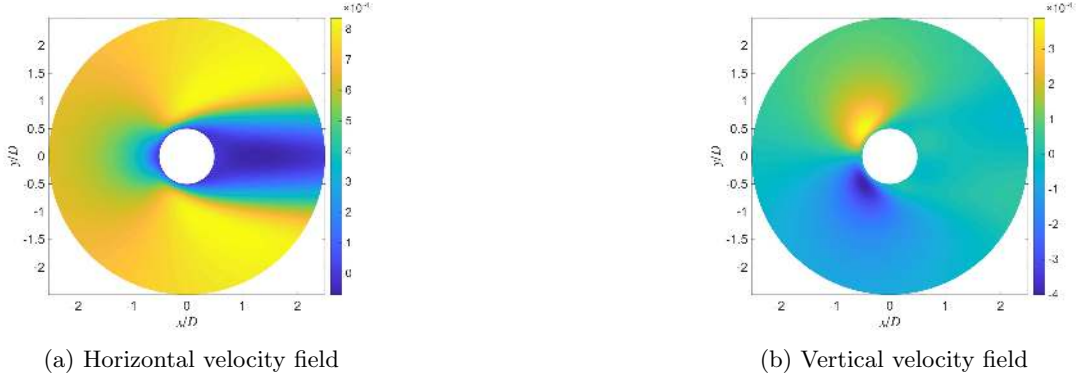


Figure 11: Cartesian velocity fields for laminar flow over a 2D cylinder.

The polar velocity fields in Figure 12 show a more insightful representation. Since we are working in a polar environment, polar coordinates tell us more about the flow. Mathematically, our boundary condition is defined with respect to polar coordinates. In this regime, the no slip condition states that the radial and tangential velocity must be zero at the cylinder wall. This is very clearly seen in the plot where the velocity is very close to zero at the wall boundary. What we would expect, then, is the presence of a boundary layer. We also see how the overall velocity of the fluid in the flow-separated region after the cylinder is very close to zero. This is due to the time-averaged erratic vortices in this region that are highly non-laminar.

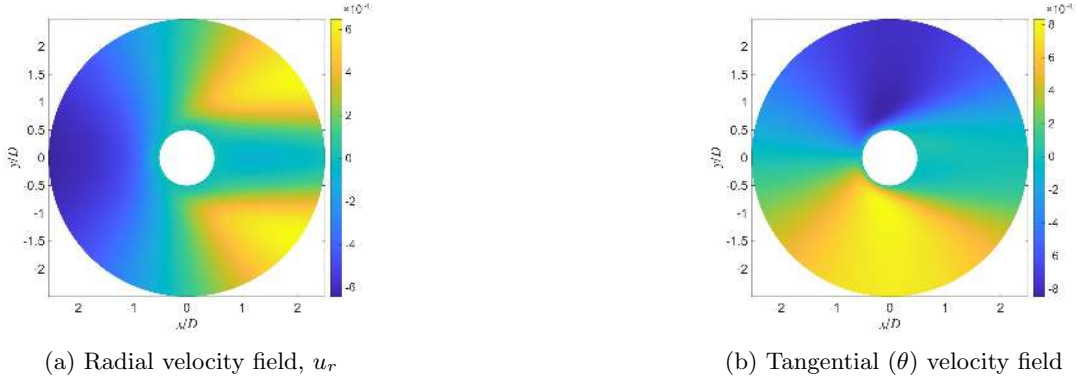


Figure 12: Polar velocity fields for laminar flow over a 2D cylinder.

To understand the properties of boundary layers in this flow regime, the most useful velocity profile would be the analogue of the x -velocity for a flat-plate, some polar velocity that runs parallel to the cylinder wall. This is the tangential velocity. Figure 13 shows the tangential velocity profile at various angles relative to the centre of the cylinder. As expected, the tangential velocity at 0° and 180° are both approximately zero. At about 45° we see a thin boundary layer developing. This boundary layer is the thickest at 90° and 270° and is slightly thinner on the side where the flow is incident to the cylinder. To determine whether this behaviour is expected, we can cross reference this with the pressure gradient in Figure 10. Initially, the flow experiences a favourable pressure gradient keeping the layer thin. As it approaches the point of lowest pressure and beyond it starts to experience an adverse pressure gradient, causing the layer to expand and change shape as it approaches the point of flow separation.

A very obvious missing feature of the velocity profile in the laminar cylinder case compared to the laminar (or turbulent) flat plate case is self-similarity. Due to the pressure gradient changing from favourable to adverse as the flow traverses over the cylinder, the physical shape of the velocity profile is changing, and is not at all self-similar.

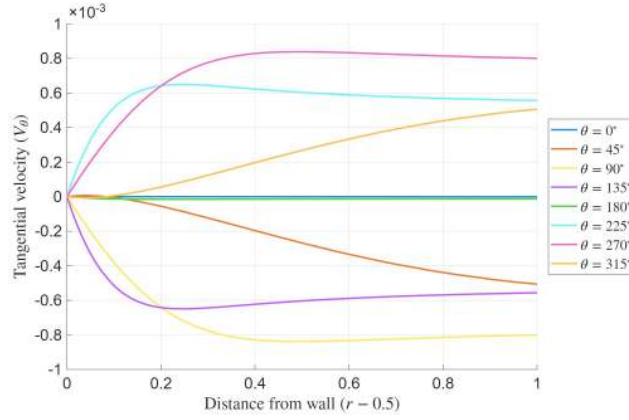


Figure 13: A plot depicting the tangential velocity profiles of the flow around a 2D cylinder for various angles about the centre.

The defining characteristic of flow in this regime is the boundary layer separation. This is a purely viscous phenomenon where the fluid no longer has enough shear stress to remain attached to the wall. As a result, the flow physically separates from the wall, causing a wake to develop behind the separation point. To find this point, Figure 14 shows the velocity gradient (proportional to the shear stress) as a function of angle from the positive x -axis. While this plot highlights the point of separation, there are some other notable features we see here. Notably, the highest friction occurs at 135° where the tangential velocity is the highest.

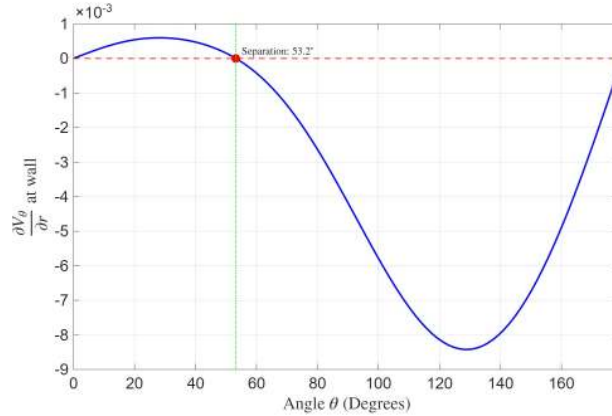


Figure 14: A plot depicting the angular point of flow separation superimposed on a plot of the wall shear stress along the cylinder's surface. Only the top half is shown, assuming symmetry over the y -axis.

The angle that flow separation occurs was found to be 53.2° from the positive x -axis, or more conventionally, 126.8° from the stagnation point. This very closely aligns with the literature of 53.8° [6].

The pressure distribution of the flow at the cylinder wall, shown in Figure 15, shows more clearly the nature of the flow over the cylinder. Most notably, similar to our contour plots, a great deal of symmetry is shown. We have a very clear stagnation point at 180° as well as two points of minimum pressure at 90° and 270° . The most significant feature of this plot, however, is the case of the wake. It is clear to see from this plot that the pressure in the wake ($<53.2^\circ$ and $>306.8^\circ$) is consistently much lower than the free-stream pressure, and the pressure gradient approaches zero. This physically represents the failure of the flow to recover pressure due to the high viscous forces in the boundary layer which inevitably lead to flow separation. As a result, this regime produces much more drag as a result of the detached flow. This increase in drag is generally the case for all detached flows in the subsonic regime.

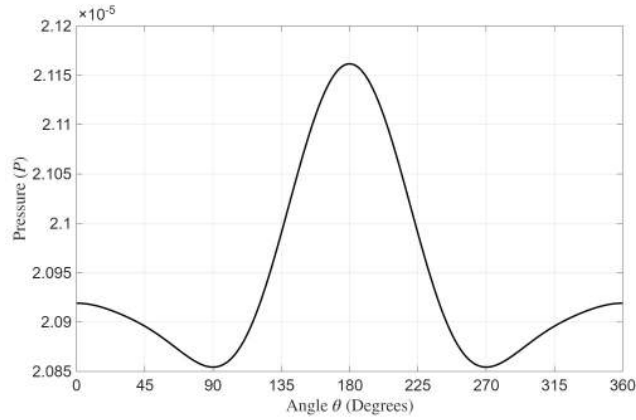


Figure 15: A plot depicting the pressure distribution at the cylinder wall as a function of angle.

In the wake region at this Reynolds number ($Re=40$) we observe two symmetric and steady vortices. This breaks down quite fast after Reynolds numbers of about 35-47 or higher⁴, where we start to see unsteady oscillating vortices known as the Karman vortex street, meaning a steady state laminar CFD solver (used here) would fail beyond this point.

3 Conclusion

We successfully used the SU2 CFD open-source code to investigate the behaviour of boundary layers across three different viscous regimes. We first analysed the simple ordered laminar flow over a flat plate by examining the thickness and friction coefficients and found these to closely align with theoretical Blasius relationships. This was found to differ greatly from the turbulent case where a thicker boundary layer was observed due to the enhanced momentum mixing. This steeper velocity gradient resulted in much higher drag than in the laminar case. We also saw that both cases were sufficiently self-similar, with the turbulent case experiencing CFD numerical breakdowns in the case surrounding $x = 0$ which caused it to differ from reality.

The flat plate was contrasted against the laminar cylinder flow regime, primarily highlighting how an adverse pressure gradient contributes to flow separation and increased drag. We found the boundary layer to detach at 53.2° from the horizontal axis which completely prevented the flow from recovering pressure, as demonstrated by the near-zero pressure gradient in the wake.

All three regimes validate literature expectations of boundary layers and provide a good basis for exploring how the flow conditions are highly dependent on the flow geometry.

4 References

1. ANSYS, Inc. What is Computational Fluid Dynamics (CFD)? 2026. Available from: <https://www.ansys.com/simulation-topics/what-is-computational-fluid-dynamics> [Accessed on: 2026 Apr 18]
2. SU2. SU2 — Multiphysics Simulation and Design Software. Version 6.1.0. 2012. Available from: <https://su2code.github.io/> [Accessed on: 2026 Apr 18]
3. Nogueira P. MAE3401 - Aerodynamics II Lecture Slides. Accessible via Moodle by staff and students of MAE3401. 2026
4. White and Majdalani. Viscous Fluid Flow, 4th Edition. McGraw Hill, 2021
5. Krishnan K, Bertram O, and Seibel O. Review of hybrid laminar flow control systems. Progress in Aerospace Sciences 2017; 93:24–52
6. Dennis SCR and Chang GZ. Numerical solutions for steady flow past a circular cylinder at Reynolds numbers up to 100. Journal of Fluid Mechanics 1970; 42:471–89. DOI: 10.1017/S0022112070001428

⁴Sources disagree. White[4] quotes 35, yet some newer studies place this value close to 47. The SU2 solver used in this task implies a steady state at $Re=40$.

5 Appendix

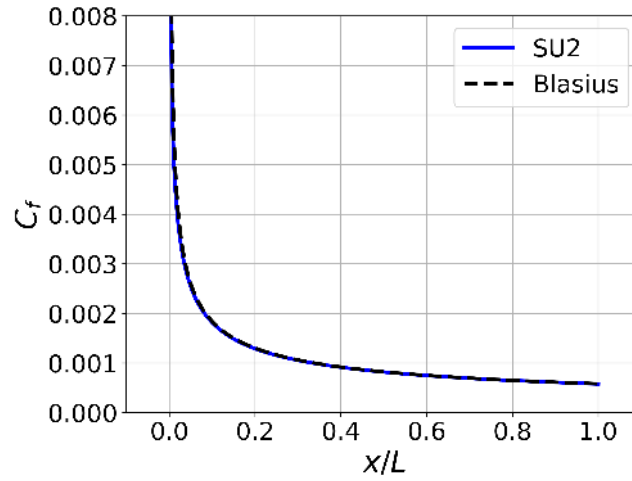


Figure 16: The predicted Blasius friction overlaid onto the output of the SU2 CFD laminar flat plate tutorial configuration which has the same configuration as our study [2].

5.1 Laminar flat plate code

```
clear all; close all; clc;
% Wrapper for SU2

set(0, 'DefaultLineLineWidth', 2, ...
    'DefaultLineMarkerSize', 10, ...
    'DefaultAxesFontSize', 18, ...
    'DefaultTextInterpreter', 'latex', ...
    'DefaultLegendInterpreter', 'latex', ...
    'DefaultLegendFontSize', 16, ...
    'DefaultFigureColor', [1 1 1])

clear all

% Run SU2 on terminal, if you have already produced a .csv, you can comment
% this if statement out to use the rest of the wrapper without running the
% CFD again.
% if ispc
%     system('SU2_CFD.exe lam_flatplate.cfg ');
% else
%     system('./SU2_CFD lam_flatplate.cfg ');
% end

%% Parameters

% Free-stream velocity
U0 = 69.1687; % m/s

% Inlet temperature
T = 297.62; % K

% Length of flatplate
total_length = 0.3048;

% Free-stream density
```

```

rho0 = 1.13235; % kg/m^3

% Free-stream dynamic viscosity
mu0 = 1.83463e-05;

generate_plots = true;

%% Main program

% We will generate the thickness plot for only the 148.81 K case first.
temperature = "T148.81";
filename = "restart_flow_lamflat_" + temperature + ".csv";

% Run importfile.m to extract data from the .csv file
[x, y, Pressure, Velocity_x, Velocity_y, Temperature, Pressure_Coefficient, Density,
    Laminar_Viscosity, Skin_Friction_Coefficient_x, Skin_Friction_Coefficient_y,
    Heat_Flux, Y_Plus] = importfile(filename);

fprintf("\nLoaded data for " + filename)

%Find number of grid points
aux=x==x(1);
nx=length(find(aux==true)); %Number of x grid points
ny=length(x)/nx; %Number of y grid points

% Reshape large column vectors from .csv into an nX x nY grid
xgrid=reshape(x,nx,ny);
ygrid=reshape(y,nx,ny);

U=reshape(Velocity_x,nx,ny);
V=reshape(Velocity_y,nx,ny);
T=reshape(Temperature,nx,ny);

xline=xgrid(1,:);
yline=ygrid(:,1);

%% Generate velocity field contour plots

if generate_plots
    figure('Visible','off')
    contourf(xgrid,ygrid,U,128,'LineStyle','none')
    xlabel('$x$')
    ylabel('$y$')
    ylim([0 0.002])
    colorbar
    saveas(gcf, "plot_u_"+temperature+".jpg")

    fprintf("\nSaved " + "plot_u_"+temperature+".jpg")

    figure('Visible','off')
    contourf(xgrid,ygrid,V,128,'LineStyle','none')
    xlabel('$x$')
    ylabel('$y$')
    ylim([0 0.002])
    colorbar
    saveas(gcf, "plot_v_"+temperature+".jpg")

    fprintf("\nSaved " + "plot_v_"+temperature+".jpg")
end

```

```

%% Calculating delta_99
% Velocity values in csv file are scaled according to U0, so we want to
% find when u = 0.99.

delta_99_values = zeros(1, length(xline)); % Initialise empty array

threshold = 0.99;

for index = 1:length(xline)

    u_profile = U(:, index);
    y_profile = ygrid(:, index);

    % Find the index where U reaches 99% of U0
    u_99_index = find(u_profile >= threshold, 1, 'last');

    % Determine the point just inside the boundary, for the linear
    % interpolation
    u_99_index_below = u_99_index + 1;

    if u_99_index_below > length(y_profile)
        delta_99_values(index) = y_profile(u_99_index);
    else
        u_above = u_profile(u_99_index);
        u_below = u_profile(u_99_index_below);
        y_above = y_profile(u_99_index);
        y_below = y_profile(u_99_index_below);

        % Calculate delta_99
        delta_99_values(index) = y_below + (y_above - y_below) * ((threshold - u_below)
            / (u_above - u_below));
    end
end

end

%% Calculating delta_star

delta_star_values = zeros(1, length(xline)); % Initialise empty array for delta_star

for index = 1:length(xline)

    u_profile = U(:, index);
    y_profile = ygrid(:, index);

    % flip arrays so y goes from wall
    u_asc = flipud(u_profile);
    y_asc = flipud(y_profile);

    % U should already be normalised but if its not can change this
    u_norm = u_asc / 1.0;

    % The integrand is simplified by the incompressible assumption and
    % the fact that velocity is already normalised
    integrand = 1 - u_norm;

    % MATLAB conveniently has a good numerical integration function
    % that can do this:
    delta_star_values(index) = trapz(y_asc, integrand);
end

```

end

%% Calculating momentum thickness theta

*% This is pretty much the exact same as above but with a slightly
% different integrand*

theta_values = **zeros**(1, **length**(xline));

for index = 1:**length**(xline)

u_profile = U(:, index);
y_profile = ygrid(:, index);

% flip arrays so y goes from wall

u_asc = **flipud**(u_profile);
y_asc = **flipud**(y_profile);

% U should already be normalised but if its not can change this

u_norm = u_asc / 1.0;

*% The integrand is simplified by the incompressible assumption and
% the fact that velocity is already normalised*

integrand = u_norm .* (1 - u_norm);

*% MATLAB conveniently has a good numerical integration function
% that can do this:*

theta_values(index) = **trapz**(y_asc, integrand);

end

%% Calculating friction coefficient

friction_coeffs = **zeros**(1, **length**(xline));

% dynamic pressure

q = 0.5 * rho0 * U0^2;

u_real = U * U0;

for index = 1:**length**(xline)

if (xline(index) > 0) *% on the flat plate only*

u_profile = u_real(:, index);
y_profile = ygrid(:, index);

u_asc = **flipud**(u_profile);
y_asc = **flipud**(y_profile);

% linear interp

dy = y_asc(2) - y_asc(1);
du = u_asc(2) - u_asc(1);

tau_wall = mu0 * (du/dy);

friction_coeffs(index) = tau_wall / q;

else

friction_coeffs(index) = 0; *% if not on plate*

end

```

end

%% Normalising the velocity profile by the streamwise boundary layer thickness to test
for self-similarity
if generate_plots
    figure("Visible","off")
    hold on

    % Find where plate starts
    plate_start_idx = find(xline >= 0, 1, 'first');

    % Test a few evenly spaced stations
    stations = round(linspace(plate_start_idx + 10, length(xline), 5));

    for idx = stations
        u_profile = U(:, idx);
        y_profile = ygrid(:, idx);

        % Normalise y-profile by delta_99
        y_norm = y_profile ./ delta_99_values(idx);

        u_norm = u_profile; % Already normalised
        plot(u_norm, y_norm, "DisplayName", sprintf('$x/L = %.3f$', xline(idx)/
            total_length))
    end

    % Formatting the plot
    xlabel('$u$')
    ylabel('$y/\delta_{99}$')
    legend('Location', 'best')
    % Zoom in to the relevant part of the boundary layer
    ylim([0 1.4])
    xlim([0 1.1])
    grid on
    hold off
    saveas(gcf, "norm_profiles_"+temperature+".jpg")

    fprintf("\nSaved " + "norm_profiles_"+temperature+".jpg")
end

%% Total drag coefficient

total_drag_coeff = 1/(max(xline)) * trapz(xline(xline>0), friction_coeffs(xline>0));

fprintf("\nTotal drag coefficient per unit length: %.8f", total_drag_coeff)

fprintf("\nCalculations done for " + temperature)

%% Generate boundary thickness plot

if generate_plots
    figure("Visible","off")
    hold on
    plot(xline/total_length, delta_99_values)
    plot(xline/total_length, delta_star_values)
    plot(xline/total_length, theta_values)
    hold off
    ylim([0 1.5*max(delta_99_values)])
    grid("on")

```



```

xlabel( '$x/L$ ')
ylabel( '$y/L$ ')
legend("$\delta_{99}$ Boundary Layer Thickness", "$\delta^*$ Displacement Thickness",
      "$\theta$ Momentum Thickness", Location="northwest")
saveas(gcf, "thickness_"+temperature+".jpg")

fprintf("\nSaved " + "thickness_"+temperature+".jpg")
end

%% Boundary thickness plot with Blasius solution overlaid

local_reynolds = zeros(1, length(xline));

for i = 1:length(xline)
    local_reynolds(i) = abs(rho0 * U0 * xline(i) / mu0);
end

blasius_delta = 4.91 * xline ./ sqrt(local_reynolds);
blasius_delta_star = 1.721 * xline ./ sqrt(local_reynolds);
blasius_theta = 0.664 * xline ./ sqrt(local_reynolds);

if generate_plots
    figure("Visible","off")
    hold on
    plot(xline/total_length, delta_99_values, 'r.', DisplayName=sprintf('$\delta_{99}$ Boundary Layer Thickness'))
    plot(xline/total_length, blasius_delta, 'r', DisplayName=sprintf('Blasius solution for $\delta_{99}$'))
    plot(xline/total_length, delta_star_values, 'b.', DisplayName=sprintf('$\delta^*$ Displacement Thickness'))
    plot(xline/total_length, blasius_delta_star, 'b', DisplayName=sprintf('Blasius solution for $\delta^*$'))
    plot(xline/total_length, theta_values, 'g.', DisplayName=sprintf('$\theta$ Momentum Thickness'))
    plot(xline/total_length, blasius_theta, 'g', DisplayName=sprintf('Blasius solution for $\theta$'))
    hold off
    ylim([0 1.5*max(delta_99_values)])
    grid("on")
    xlabel( '$x/L$ ')
    ylabel( '$y/L$ ')
    legend(Location="northwest")
    saveas(gcf, "thickness-blasius_"+temperature+".jpg")

    fprintf("\nSaved " + "thickness-blasius_"+temperature+".jpg")
end

%% Generate friction coefficient plot

if generate_plots
    figure("Visible","off")
    plot(xline/total_length, friction_coeffs)
    xlim([0 inf])
    ylim([0 4*median(friction_coeffs)])
    grid("on")
    xlabel( '$x/L$ ')
    ylabel( '$c_f$ ')
    saveas(gcf, "friction_"+temperature+".jpg")
end

```

```

    fprintf("\nSaved " + "friction_" + temperature + ".jpg")
end

%% Temperature profiles at specific x location

if generate_plots
    figure("Visible","off")
    hold on

    temperatures = ["T148.81", "T297.62", "T350"];

    % Pick an arbitrary target x-value to compare
    target_x = 0.15;

    for i = 1:length(temperatures)
        temp_label = temperatures(i);
        filename = "restart_flow_lamflat_" + temp_label + ".csv";

        % Load the data for this specific case
        [x_data, y_data, ~, ~, ~, T_data, ~, ~, ~, ~, ~, ~] = importfile(filename);
        fprintf("\nLoaded data for " + filename)

        % Find grid dimensions (same as before)
        nx_points = length(find(x_data == x_data(1)));
        ny_points = length(x_data) / nx_points;

        % Reshape the coordinates and temperature data
        xgrid = reshape(x_data, nx_points, ny_points);
        ygrid = reshape(y_data, nx_points, ny_points);
        T_grid = reshape(T_data, nx_points, ny_points);

        % Find nearest x value
        xline = xgrid(1, :);
        [~, target_idx] = min(abs(xline - target_x));

        y_profile = ygrid(:, target_idx);
        t_profile = T_grid(:, target_idx);

        plot(t_profile * 297.62, y_profile, "DisplayName", temp_label + " Wall")
    end

    % Formatting the plot
    xlabel('Temperature (K)')
    ylabel('$y$ (m)')
    legend('Location', 'northwest')
    ylim([0 0.002]) % only show area of interest
    grid on
    hold off

    saveas(gcf, "temp_profiles.jpg")
    fprintf("\nSaved temp_profiles.jpg")
end

fprintf("\n\n")

```

5.2 Turbulent flat plate code

```

clear all; close all; clc;
% Wrapper for SU2

```

```

set(0,'DefaultLineLineWidth',2,...
    'DefaultLineMarkerSize',10,...
    'DefaultAxesFontSize',18,...
    'DefaultTextInterpreter','latex',...
    'DefaultLegendInterpreter','latex',...
    'DefaultLegendFontSize',16,...
    'DefaultFigureColor',[1 1 1])

clear all

% Run SU2 on terminal, if you have already produced a .csv, you can comment
% this if statement out to use the rest of the wrapper without running the
% CFD again.
%if ispc
%    system('SU2-CFD.exe turb_flatplate.cfg');
%else
%    system('./SU2-CFD turb_flatplate.cfg');
%end

%% Configuration
generate_plots = true;

rho0 = 1.32905;

U0 = 69.4448;

mu0 = 1.84592e-05;

total_length = 2.0;

%%

% Run importfile.m to extract data from the .csv file
filename = 'restart_flow_turbflatplate.csv';
[x, y, Pressure, Velocity_x, Velocity_y, Temperature, Pressure_Coefficient, Density,
    Laminar_Viscosity, Skin_Friction_Coefficient_x, Skin_Friction_Coefficient_y,
    Heat_Flux, Y_Plus] = importfile(filename);

fprintf("\nLoaded data for " + filename)

%Find number of grid points
aux=x==x(1);
ny=length(find(aux==true)); %Number of x grid points
nx=length(x)/ny; %Number of y grid points

% Reshape large column vectors from .csv into an nX x nY grid
xgrid=reshape(x,ny,nx);
ygrid=reshape(y,ny,nx);

U=reshape(Velocity_x,ny,nx);
V=reshape(Velocity_y,ny,nx);
T=reshape(Temperature,ny,nx);

xline=xgrid(1,:);
yline=ygrid(:,1);

%% Generate velocity field contour plots

```

```

if generate_plots
    figure( 'Visible ', 'off ')
    contourf(xgrid,ygrid,U,128,'LineStyle','none')
    xlabel( '$x$ ')
    ylabel( '$y$ ')
    ylim([0 0.02])
    colorbar
    saveas(gcf, "plot_u_turbflat.jpg")

    fprintf("\nSaved " + "plot_u_turbflat.jpg")

    figure( 'Visible ', 'off ')
    contourf(xgrid,ygrid,V,128,'LineStyle','none')
    xlabel( '$x$ ')
    ylabel( '$y$ ')
    ylim([0 0.02])
    colorbar
    saveas(gcf, "plot_v_turbflat.jpg")

    fprintf("\nSaved " + "plot_v_turbflat.jpg")
end

%% Calculating delta_99
% Velocity values in csv file are scaled according to U0, so we want to
% find when u = 0.99.

delta_99_values = zeros(1, length(xline)); % Initialise empty array

threshold = 0.99;

for index = 1:length(xline)

    u_profile = U(:, index);
    y_profile = ygrid(:, index);

    % Find the index where U reaches 99% of U0
    u_99_index = find(u_profile >= threshold, 1, 'first');

    % Determine the point just inside the boundary, for the linear
    % interpolation
    u_99_index_below = u_99_index - 1;

    if u_99_index_below < 1
        delta_99_values(index) = y_profile(u_99_index);
    else
        u_above = u_profile(u_99_index);
        u_below = u_profile(u_99_index_below);
        y_above = y_profile(u_99_index);
        y_below = y_profile(u_99_index_below);

        % Calculate delta_99
        delta_99_values(index) = y_below + (y_above - y_below) * ((threshold - u_below)
            / (u_above - u_below));
    end
end

end

%% Calculating delta_star

```

```

delta_star_values = zeros(1, length(xline)); % Initialise empty array for delta_star

for index = 1:length(xline)

    u_profile = U(:, index);
    y_profile = ygrid(:, index);

    % U should already be normalised but if its not can change this
    u_norm = u_profile / 1.0;

    % The integrand is simplified by the incompressible assumption and
    % the fact that velocity is already normalised
    integrand = 1 - u_norm;

    % MATLAB conveniently has a good numerical integration function
    % that can do this:
    delta_star_values(index) = trapz(y_profile, integrand);

end

%% Calculating momentum thickness theta
% This is pretty much the exact same as above but with a slightly
% different integrand

theta_values = zeros(1, length(xline));

for index = 1:length(xline)

    u_profile = U(:, index);
    y_profile = ygrid(:, index);

    % U should already be normalised but if its not can change this
    u_norm = u_profile / 1.0;

    % The integrand is simplified by the incompressible assumption and
    % the fact that velocity is already normalised
    integrand = u_norm .* (1 - u_norm);

    % MATLAB conveniently has a good numerical integration function
    % that can do this:
    theta_values(index) = trapz(y_profile, integrand);

end

%% Calculating friction coefficient

friction_coeffs = zeros(1, length(xline));

% dynamic pressure
q = 0.5 * rho0 * U0^2;

u_real = U * U0;

for index = 1:length(xline)

    if (xline(index) > 0) % on the flat plate only
        u_profile = u_real(:, index);
        y_profile = ygrid(:, index);
    end

```

```

    % linear interp
    dy = y_profile(2) - y_profile(1);
    du = u_profile(2) - u_profile(1);

    tau_wall = mu0 * (du/dy);

    friction_coeffs(index) = tau_wall / q;
else
    friction_coeffs(index) = 0; % if not on plate
end
end

%% Normalising the velocity profile by the streamwise boundary layer thickness to test
for self-similarity
if generate_plots
    figure("Visible","off")
    hold on

    % Find where plate starts
    plate_start_idx = find(xline >= 0, 1, 'first');

    % Test a few evenly spaced stations
    stations = round(linspace(plate_start_idx + 10, length(xline), 5));

    for idx = stations
        u_profile = U(:, idx);
        y_profile = ygrid(:, idx);

        % Normalise y-profile by delta_99
        y_norm = y_profile ./ delta_99_values(idx);

        u_norm = u_profile; % Already normalised
        plot(u_norm, y_norm, "DisplayName", sprintf('$x=-%.3f$', xline(idx)/total_length))
    end

    % Formatting the plot
    xlabel('$u$')
    ylabel('$y/-\delta_{99}$')
    legend('Location', 'best')
    % Zoom in to the relevant part of the boundary layer
    ylim([0 1.4])
    xlim([0 1.1])
    grid on
    hold off
    saveas(gcf, "norm_profiles_turbflat.jpg")

    fprintf("\nSaved " + "norm_profiles_turbflat.jpg")
end

%% Total drag coefficient

plate_idx = xline > 0;
total_drag_coeff = (1/max(xline)) * trapz(xline(plate_idx), friction_coeffs(plate_idx));

fprintf("\nTotal drag coefficient per unit length: %.8f", total_drag_coeff)

%% Generate boundary thickness plot

```

```

if generate_plots
    figure("Visible","off")

    yyaxis left
    plot(xline/total_length, delta_99_values, DisplayName=sprintf('%\\delta_{99}$-
        Boundary-Layer-Thickness'))
    ylim([0 inf])

    yyaxis right
    hold on
    plot(xline/total_length, delta_star_values, DisplayName=sprintf('%\\delta^{*}$-
        Displacement-Thickness'))
    plot(xline/total_length, theta_values, DisplayName=sprintf('%\\theta$-Momentum-
        Thickness'))
    hold off
    ylim([0 inf])

    grid("on")
    xlabel('$x/L$')
    ylabel('$y/L$')
    legend(Location="northwest")
    saveas(gcf, "thickness_turbflat.jpg")

    fprintf("\nSaved " + "thickness_turbflat.jpg")
end

%% Generate friction coefficient plot

if generate_plots
    figure("Visible","off")
    plot(xline/total_length, friction_coeffs)
    xlim([0 inf])
    ylim([0 3*median(friction_coeffs)])
    grid("on")
    xlabel('$x/L$')
    ylabel('$C_f$')
    saveas(gcf, "friction_turbflat.jpg")

    fprintf("\nSaved " + "friction_turbflat.jpg")
end

fprintf("\n\n")

```

5.3 Laminar cylinder code

```

clear all; close all; clc;
% Wrapper for SU2

set(0, 'DefaultLineLineWidth', 2, ...
    'DefaultLineMarkerSize', 10, ...
    'DefaultAxesFontSize', 18, ...
    'DefaultTextInterpreter', 'latex', ...
    'DefaultLegendInterpreter', 'latex', ...
    'DefaultLegendFontSize', 16, ...
    'DefaultFigureColor', [1 1 1])

clear all

% Run SU2 on terminal, if you have already produced a .csv, you can comment

```

```

% this if statement out to use the rest of the wrapper without running the
% CFD again.
% if ispc
%     system('SU2-CFD.exe lam_cylinder.cfg ');
% else
%     system('./SU2-CFD lam_cylinder.cfg ');
% end

%% Configuration
generate_plots = true;

machnumber = 0.1;

% Free-stream temperature
T0 = 288.15;

% Reynold's number
reynolds = 40;

%%

% Run importfile.m to extract data from the .csv file
filename = "restart_flow_lamcyl.csv";
[x, y, Pressure, Velocity_x, Velocity_y, Temperature, Pressure_Coefficient, Density,
    Laminar_Viscosity, Skin_Friction_Coefficient_x, Skin_Friction_Coefficient_y,
    Heat_Flux, Y_Plus] = importfile(filename);

fprintf("\nLoaded data for " + filename)

% interpolation into polar mesh
nr=200;
nt=200;
r=linspace(0,2,nr)+0.5;
theta=linspace(0,2*pi,nt);

[RR,TT]=meshgrid(r,theta);

yi=RR(:).*sin(TT(:));
xi=RR(:).*cos(TT(:));

xi=reshape(xi,nr,nt);
yi=reshape(yi,nr,nt);

P=griddata(x-0.5,y,Pressure,xi,yi);
P=reshape(P,nr,nt);

U=griddata(x-0.5,y,Velocity_x,xi,yi);
U=reshape(U,nr,nt);

V=griddata(x-0.5,y,Velocity_y,xi,yi);
V=reshape(V,nr,nt);

%% Generate velocity and pressure field contour plots

if generate_plots
    figure("Visible","off")
    contourf(xi,yi,U,64,'LineStyle','none')
    axis equal
    xlabel('$x/D$')

```



```

ylabel( '$y/D$' )
colorbar
saveas(gcf, "plot_u_lamcyl.jpg")

fprintf("\nSaved " + "plot_u_lamcyl.jpg")

figure("Visible","off")
contourf(xi,yi,V,64,'LineStyle','none')
axis equal
xlabel( '$x/D$' )
ylabel( '$y/D$' )
colorbar
saveas(gcf, "plot_v_lamcyl.jpg")

fprintf("\nSaved " + "plot_v_lamcyl.jpg")

figure("Visible","off")
contourf(xi,yi,P,64,'LineStyle','none')
axis equal
xlabel( '$x/D$' )
ylabel( '$y/D$' )
colorbar
saveas(gcf, "plot_p_lamcyl.jpg")

fprintf("\nSaved " + "plot_p_lamcyl.jpg")
end

%% Polar coordinate system contour plot

% We already have the meshgrid of radii and theta angles that were given
Vr = U .* cos(TT) + V .* sin(TT);
Vtheta = -U .* sin(TT) + V .* cos(TT);

if generate_plots
    figure("Visible","off")
    contourf(xi,yi,Vr,128,"LineStyle","none")
    axis equal
    colorbar
    xlabel( '$x/D$' )
    ylabel( '$y/D$' )
    saveas(gcf, "plot_Vr.jpg")

    fprintf("\nSaved " + "plot_Vr.jpg")

    figure("Visible","off")
    contourf(xi,yi,Vtheta,128,"LineStyle","none")
    axis equal
    colorbar
    xlabel( '$x/D$' )
    ylabel( '$y/D$' )
    saveas(gcf, "plot_Vtheta.jpg")

    fprintf("\nSaved " + "plot_Vtheta.jpg")

    fprintf("\nThe tangential velocity component is better to analyse the boundary layer
        .")
end

%% Tangential velocity as a function of radius for several theta

```

```

if generate_plots
    figure("Visible","off")
    hold on
    colororder("glow12") % MATLAB doesn't have enough default colours so use a colormap

    angles = linspace(0, 2*pi, 9);
    angles(end) = []; % Remove 360 degree entry as its same as zero

    r_wall = r - 0.5;

    for i = 1:length(angles)

        % Find row index for closest theta value
        [~, t_idx] = min(abs(theta - angles(i)));

        vtheta_profile = Vtheta(t_idx, :);

        plot(r_wall, vtheta_profile, 'DisplayName', sprintf('$\\theta = %d^\\circ$',
            rad2deg(angles(i))))
    end

    % Formatting the plot
    xlabel('Distance from wall ($r - 0.5$)')
    ylabel('Tangential velocity ($V_\\theta$)')
    legend('Location', 'eastoutside')
    grid on

    % Zoom in on focus area
    xlim([0 1.0])
    hold off

    saveas(gcf, "plot_thetaprofiles.jpg");

    fprintf("\nSaved plot_thetaprofiles.jpg")
end

%% Point of flow separation
% Occurs at zero wall shear stress, or simply when the normal derivative of
% tangential velocity is zero.

% Method using finite differences
% I initially used a first order finite difference, however my result
% varied from the literature. There is no reason in this case I cannot use
% a better approximation and get a result that should align better with
% literature. Hence, this is a second order finite difference.
dr = r(2) - r(1);
dvtheta_dr = (-3*Vtheta(:,1) + 4*Vtheta(:,2) - Vtheta(:,3)) / (2*dr);

% Since the problem is symmetric about x, let's only look at the top half:
top_half_idx = find(theta <= pi);

theta_top = theta(top_half_idx);
derivative_top = dvtheta_dr(top_half_idx);

% The derivative likely won't equal exactly zero but let's look where it
% flips signs (+ to - or - to +)
sign_flip = find(abs(diff(sign(derivative_top))) > 0);

if isempty(sign_flip)

```

```

    fprintf("\nNo shear stress sign changes found.")
else
    separation_index = sign_flip(end); % In the case of multiple roots, use the last one
    (representing the flow detachment)

    % Use linear interpolation same as what I used before
    t1 = theta_top(separation_index);
    t2 = theta_top(separation_index + 1);
    d1 = derivative_top(separation_index);
    d2 = derivative_top(separation_index + 1);

    theta_separation = t1 - d1 * (t2 - t1) / (d2 - d1);
    fprintf('\nFlow separation occurs at exactly: %.2f-degrees.', rad2deg(
        theta_separation));
    fprintf('\nConvention is to write it with respect to the upstream stagnation point: -
        %.2f-degrees.', 180-rad2deg(theta_separation));
end

% plot the wall gradient vs theta to visualize the zero-crossing
if generate_plots
    figure('Visible','off')
    plot(rad2deg(theta_top), derivative_top, 'b-', 'LineWidth', 2)
    hold on
    yline(0, 'r--', 'LineWidth', 1.5) % Plot the zero-stress line

    if ~isempty(sign_flip)
        % Mark the exact separation point
        plot(rad2deg(theta_separation), 0, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r'
        )
        xline(rad2deg(theta_separation), 'g:', 'LineWidth', 1.5)
        text(rad2deg(theta_separation)*1.03, max(derivative_top)*0.4, sprintf('---
            Separation: %.1f^\circ', rad2deg(theta_separation)), 'FontSize', 12)
    end

    xlabel('Angle-\theta (Degrees)')
    ylabel('$\frac{\partial V}{\partial r}$ at wall')
    xlim([0 max(rad2deg(theta_top))])
    grid on
    hold off
    saveas(gcf, "plot_flowseparation.jpg")

    fprintf("\nSaved plot_flowseparation.jpg")
end

%% Pressure distribution as a function of theta at wall boundary

% Just need the pressure at the wall
P_wall = P(:, 1);

theta_deg = rad2deg(theta);

if generate_plots

    figure('Visible','off')
    plot(theta_deg, P_wall, 'k-', 'LineWidth', 2)

    xlabel('Angle-\theta (Degrees)')
    ylabel('Pressure-(P)')

```

```

    % standardise axes
    xlim([0 360])
    xticks(0:45:360)
    grid on
    saveas(gcf, "plot-wall-pressure.jpg")

    fprintf("\nSaved plot-wall-pressure.jpg")
end

fprintf("\n\n")

```